

Report: Bangalore Restaurants Analysis

Data Mining & Visualization A (2024)

Cika Rahmannia Febrianti (5003221081)

Using the Zomato Bangalore restaurant dataset from Kaggle, students are tasked with performing a comprehensive analysis to gain insights into the restaurant data. The analysis should be structured as follows:

- Data Pre-processing
- Research Questions
- Methodology
- Code Implementation
- Analysis and Results
- Interpretation and Conclusion

Data Description

Variabel	Deskripsi
URL	The restaurant's website URL
address	address
name	Restaurant's name
online_orders	Take online orders or not
book_table	Offers reservation or not
rate	Rating (-/5)
votes	Number of vote received
phone	Restaurant's phone number
location	The neighborhood in which the resto is located
rest_type	Type of resto
dish_liked	Dish popular
cuisines	Cuisines available
approx_cost (for two people)	Estimated cost for two people
review_list	Review given by users
Menu_item	Menu
listed_in(type)	Type service provided by resto
listed_in(city)	City

Data Pre-processing

loading data

```
import pandas as pd
import numpy as np
import seaborn as sns
import scipy.stats as stats
import matplotlib.pyplot as plt

df=pd.read_csv('D:\Data Mining & Visualisasi\zomato.csv')
df.head()
```

output (menampilkan 5 data teratas)

	url	address	name	online_order	book_table	rate	votes	phone
0	https://www.zomato.com/bangalore/jalsa-banashankari	942, 21st Main Road, 2nd Stage, Banashankari, ...	Jalsa	Yes	Yes	4.1/5	775	42297555\r\n+91 974377223

memeriksa bentuk dataset (baris x kolom)

```
df.shape
```

```
(51717, 17)
```

memeriksa informasi setiap variabel/atribut

```
df.info() ## sudah mengandung dtypes

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51717 entries, 0 to 51716
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   url                                    51717 non-null  object
1   address                               51717 non-null  object
2   name                                  51717 non-null  object
3   online_order                          51717 non-null  object
4   book_table                            51717 non-null  object
5   rate                                  43942 non-null  object
6   votes                                 51717 non-null  int64
7   phone                                 50509 non-null  object
8   location                              51696 non-null  object
9   rest_type                             51490 non-null  object
10  dish_liked                            23639 non-null  object
11  cuisines                              51672 non-null  object
12  approx_cost(for two people)           51371 non-null  object
13  reviews_list                          51717 non-null  object
14  menu_item                             51717 non-null  object
15  listed_in(type)                       51717 non-null  object
16  listed_in(city)                       51717 non-null  object
dtypes: int64(1), object(16)
memory usage: 6.7+ MB
```

~ ada beberapa kolom yang tidak penting untuk digunakan, seperti: URL, phone, location (sudah ada di listed_in(city)), menu_item kosong, dan review_list dipilih tidak digunakan karena sudah ada rating.

drop unnecessary column

```
df.drop(columns=['url', 'address', 'phone', 'reviews_list', 'menu_item'], inplace=True)
df.head()
```

rename nama kolom

```
df.rename(columns={
    'listed_in(city)': 'city_loc',
    'approx_cost(for two people)': 'costfor2',
    'listed_in(type)': 'type',
    inplace=True)
df
```

output setelah drop kolom

	name	online_order	book_table	rate	votes	location	rest_type	dish_liked	cuisines	costfor2	type	city_loc
0	Jalsa	Yes	Yes	4.1/5	775	Banashankari	Casual Dining	Pasta, Lunch Buffet, Masala Papad, Paneer Laja...	North Indian, Mughlai, Chinese	800	Buffet	Banashankari
1	Spice Elephant	Yes	No	4.1/5	787	Banashankari	Casual Dining	Momos, Lunch Buffet, Chocolate Nirvana, Thai G...	Chinese, North Indian, Thai	800	Buffet	Banashankari
2	San Churro Cafe	Yes	No	3.8/5	918	Banashankari	Cafe, Casual Dining	Churros, Cannelloni, Minestrone Soup, Hot Choc...	Cafe, Mexican, Italian	800	Buffet	Banashankari
3	Addhuri Udupi Bhojana	No	No	3.7/5	88	Banashankari	Quick Bites	Masala Dosa	South Indian, North Indian	300	Buffet	Banashankari
4	Grand Village	No	No	3.8/5	166	Basavanagudi	Casual Dining	Panipuri, Gol Gappe	North Indian, Rajasthani	600	Buffet	Banashankari

mencari duplicate row

```
sum(df.duplicated())
```

108

drop duplicate row, cek duplikasi, cek bentuk data terbaru

```
df=df.drop_duplicates()
```

```
sum(df.duplicated())
```

0

```
df.shape
```

(51609, 12)

drop salah satu location dan rename lainnya

```
df.drop(columns=['location'], inplace=True)
```

```
df.rename(columns={'city_loc': 'location'}, inplace=True)
df
```

cek null count dan tipe data

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 51609 entries, 0 to 51716
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0    name        51609 non-null  object
1   online_order  51609 non-null  object
2   book_table   51609 non-null  object
3    rate        43854 non-null  object
4   votes        51609 non-null  int64
5   rest_type    51382 non-null  object
6   dish_liked   23595 non-null  object
7   cuisines     51564 non-null  object
8   costfor2     51265 non-null  object
9    type        51609 non-null  object
10  location     51609 non-null  object
dtypes: int64(1), object(10)
memory usage: 4.7+ MB
```

~ non-null count untuk setiap atribut tidak sama

data describe

```
df.describe(include='all')
```

	name	online_order	book_table	rate	votes	rest_type	dish_liked	cuisines	costfor2	type	location
count	51609	51609	51609	43854	51609.000000	51382	23595	51564	51265	51609	51609
unique	8792	2	2	64	NaN	93	5271	2723	70	7	30
top	Cafe Coffee Day	Yes	No	NEW	NaN	Quick Bites	Biryani	North Indian	300	Delivery	BTM
freq	96	30361	45193	2195	NaN	19096	182	2907	7558	25867	3266
mean	NaN	NaN	NaN	NaN	283.283361	NaN	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	803.282771	NaN	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	0.000000	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	7.000000	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	41.000000	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	198.000000	NaN	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	16832.000000	NaN	NaN	NaN	NaN	NaN	NaN

~ data kuantitatif hanya pada 'vote', sedangkan lainnya kualitatif dengan acuan frekuensi.

cek missing value

```
print(df.isnull().sum())

name          0
online_order  0
book_table    0
rate         7755
votes         0
rest_type     227
dish_liked   28014
cuisines      45
costfor2     344
type          0
location      0
dtype: int64
```

~ terdapat banyak atribut dengan missing value (>0)

cek persentase missing value

```
(df.isna().sum()/len(df))*100

name          0.000000
online_order  0.000000
book_table    0.000000
rate         15.026449
votes         0.000000
rest_type     0.439846
dish_liked    54.281230
cuisines      0.087194
costfor2      0.666550
type          0.000000
location      0.000000
dtype: float64
```

~ atribut 'dish_liked' memiliki missing value besar (54.28%) sehingga best option to handle adalah dengan remove column. Jika dilakukan imputasi, maka akan terdapat bias yang besar.

cek jumlah value unik

```
df['dish_liked'].value_counts()
```

```
dish_liked
Biryani 182
Chicken Biryani 73
Friendly Staff 69
Waffles 68
Paratha 57
...
Banana Cake, Ginger Lemon Tea, Poha, Mango Shake, Masala Chai, Paneer Sandwich, Vada Pav 1
Butter Chicken, Shawarma Roll, Chicken Shawarama, Chicken Grill, Rolls, Al Faham Chicken, Biryani 1
Filter Coffee, Sandwich, Bonda, Vada, Masala Dosa, Salad, Aloo Curry 1
Burgers, Fries, Jumbo Royale Burger, Salads, Peri Peri Chicken Salad, Potato Wedges, Rolls 1
Paratha, Dal Makhani, Lassi, Naan, Veg Thali, Chole, Kulcha 1
Name: count, Length: 5271, dtype: int64
```

drop kolom 'dish_liked'

```
df.drop(columns=['dish_liked'],inplace=True)
df
```

cek jumlah nilai unik

```
df['rate'].value_counts()
```

```
rate
NEW 2195
3.9/5 2095
3.8/5 2020
3.7/5 2009
3.9 /5 1872
...
2.2 /5 7
2.0 /5 7
2.0/5 4
1.8 /5 3
1.8/5 2
Name: count, Length: 64, dtype: int64
```

~ atribut 'rate' memiliki missing value cukup besar (15%). Ada value bukan numerik, melainkan 'NEW' dan '-'. Sehingga perlu dilakukan cleaning untuk mengubah value non-numerik tersebut ke np.nan.

cleaning non-numerik value

```
def cleaning_rate(value):
    if(value=='NEW' or value=='-'):
        return np.nan
    else:
        return float(str(value).split('/')[0])

df['rate']=df['rate'].apply(cleaning_rate)
```

output bebas non-numerik

```
df['rate'].value_counts()
```

```
rate
3.9    3967
3.8    3870
3.7    3814
3.6    3311
4.0    3180
4.1    2942
3.5    2781
3.4    2472
3.3    2305
4.2    2179
3.2    1867
4.3    1686
3.1    1561
4.4    1137
3.0    1022
2.9     801
4.5     653
2.8     600
2.7     307
4.6     299
2.6     260
4.7     167
2.5     101
2.4      70
4.8      66
4.9      55
2.3      51
2.2      26
2.1      24
2.0      11
1.8       5
Name: count, dtype: int64
```

cek missing value/NaN

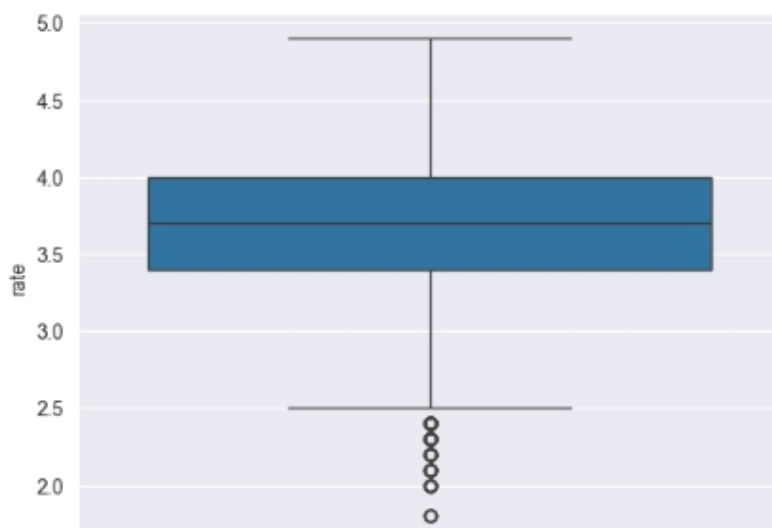
```
print(df.isnull().sum())
```

```
name           0
online_order   0
book_table     0
rate          10019
votes          0
rest_type      227
cuisines       45
costfor2       344
type           0
location       0
dtype: int64
```

cek menggunakan box plot untuk melihat adanya outlier

```
sns.set_style("darkgrid")
sns.boxplot(df['rate'])
```

<Axes: ylabel='rate'>



imputing with median

```
# There is a presence of outliers in the data so it's a good idea to impute median in place of nan
df['rate']=df['rate'].fillna(df['rate'].median())
df['rate'].isna().sum()

C:\Users\cikar\AppData\Local\Temp\ipykernel_14096\2281000533.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing-advanced.html
df['rate']=df['rate'].fillna(df['rate'].median())
```

0

~ adanya missing value/NaN pada data 'rate' sebesar 15% tidak dapat dilakukan remove column. Dilakukan proses imputation median. Imputasi median cenderung digunakan jika ada outlier dalam dataset. It helps maintain a more representative measure, sehingga kualitas analisis akan bagus.

cek missing value

```
print(df.isnull().sum())
```

```
name          0
online_order  0
book_table     0
rate          0
votes         0
rest_type     227
cuisines       45
costfor2      344
type          0
location      0
dtype: int64
```

cek jumlah nilai unik pada rest_type

```
df['rest_type'].value_counts()
```

```
rest_type
Quick Bites          19096
Casual Dining        10309
Cafe                 3727
Delivery             2600
Dessert Parlor       2260
...
Dessert Parlor, Kiosk      2
Food Court, Beverage Shop  2
Dessert Parlor, Food Court  2
Sweet Shop, Dessert Parlor  1
Quick Bites, Kiosk        1
Name: count, Length: 93, dtype: int64
```

menampilkan top 10 jenis resto yang paling sering ada dalam data

```
# Menghitung jumlah kemunculan setiap jenis restoran
top_rest_types = df['rest_type'].value_counts().head(10)
print(top_rest_types)
```

```
rest_type
Quick Bites          19096
Casual Dining        10309
Cafe                 3727
Delivery             2600
Dessert Parlor       2260
Takeaway, Delivery   2031
Casual Dining, Bar    1148
Bakery               1140
Beverage Shop        865
Bar                  697
Name: count, dtype: int64
```

~ ada banyak kategori dalam rest_type. Data belum bersih karena masih ada value yang dipisahkan koma di dalamnya. Sehingga dilakukan kombinasi agar kategori pada rest_type semakin menurun.

combine rest_type

```
df['rest_type'].value_counts()
newrest_type=pd.DataFrame(df['rest_type'].value_counts())
newrest_type[newrest_type['count']<1000]
newrest_type=newrest_type[newrest_type['count']<1000]
newrest_type
```

	count
rest_type	
Beverage Shop	865
Food Court	621

```
def combine_restt(value):
    if (value in newrest_type.index):
        return 'Others'
    else:
        return value
df['rest_type']=df['rest_type'].apply(combine_restt)
df['rest_type']

C:\Users\cikar\AppData\Local\Temp\ipykernel_14096\694883
A value is trying to be set on a copy of a slice from a
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pyd
-versus-a-copy
df['rest_type']=df['rest_type'].apply(combine_restt)

0      Casual Dining
1      Casual Dining
2      Others
3      Quick Bites
4      Casual Dining
...
51712      Bar
51713      Bar
51714      Bar
51715      Bar
51716      Others
Name: rest_type, Length: 51382, dtype: object
```

~ data dengan count < 1000 akan dimasukkan ke kategori 'others'

drop missing value

```
print(df.isnull().sum())
```

```
name      0
online_order  0
book_table  0
rate      0
votes     0
rest_type 227
cuisines  45
costfor2  344
type      0
location  0
dtype: int64
```

```
df.dropna(subset=['rest_type'],inplace=True)
```

~ only subset 'rest_type' yang akan diperiksa dan nantinya hasil perubahan akan langsung dibuat pada data original.

remove comma

```
# Removing commas from the values
df['costfor2'].str.replace(",","")
df['costfor2']=df['costfor2'].str.replace(",","")
```

convert to numeric agar data type 'object' dapat diubah ke float

```
# Convert 'costfor2' to numeric
df['costfor2']=pd.to_numeric(df['costfor2'],errors='coerce')
```

~ any non-numeric will be replaced by NaN

cek data info

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 51609 entries, 0 to 51716
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   name            51609 non-null  object
 1   online_order    51609 non-null  object
 2   book_table      51609 non-null  object
 3   rate            51609 non-null  float64
 4   votes          51609 non-null  int64
 5   rest_type       51382 non-null  object
 6   cuisines        51564 non-null  object
 7   costfor2        51265 non-null  float64
 8   type            51609 non-null  object
 9   location        51609 non-null  object
dtypes: float64(2), int64(1), object(7)
memory usage: 4.3+ MB
```

~ tipe data untuk atribut kuantitatif sudah benar.

replace NaN value

```
# Nan values are replaced by median of that type of restaurant
df['costfor2'].fillna().sum()
```

```
321
```

cleaning rest_type

```
df['rest_type'].unique()
```

```
array(['Casual Dining', 'Others', 'Quick Bites', 'Cafe', 'Delivery',
       'Dessert Parlor', 'Bakery', 'Takeaway, Delivery',
       'Casual Dining, Bar'], dtype=object)
```

```
def cleaning_resttype(value):
    if (value=='Takeaway, Delivery'):
        return 'Delivery'
    elif (value=='Casual Dining, Bar'):
        return 'Bar'
    else:
        return value

df['rest_type'].apply(cleaning_resttype).value_counts()
df['rest_type']=df['rest_type'].apply(cleaning_resttype)
```

calculate median

```
# Calculate the median costfor2 for each rest_type and round it to 0 decimal places
median_cost = round(df.groupby('rest_type')['costfor2'].median(), 0)
median_cost
```

```
rest_type
Bakery      400.0
Bar         1200.0
Cafe        600.0
Casual Dining  700.0
Delivery     400.0
Dessert Parlor  300.0
Others      500.0
Quick Bites  300.0
Name: costfor2, dtype: float64
```

imputasi NaN dengan median dan cek missing value lagi

```
df['costfor2'].fillna(df['rest_type'].map(median_cost),inplace=True)
```

C:\Users\cikar\AppData\Local\Temp\ipykernel_14096\2726089572.py:1: Se
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: [https://pandas.pydata.org/panda
returning-a-view-versus-a-copy](https://pandas.pydata.org/panda
returning-a-view-versus-a-copy)

```
df['costfor2'].fillna(df['rest_type'].map(median_cost),inplace=True)
```

```
df['costfor2'].isna().sum()
```

```
0
```

cek missing value

```
print(df.isnull().sum())
```

```
name          0  
online_order  0  
book_table    0  
rate          0  
votes         0  
rest_type     0  
cuisines      24  
costfor2      0  
type          0  
location      0  
dtype: int64
```

cek jumlah nilai unik

```
df['cuisines'].value_counts()
```

```
cuisines  
North Indian          2893  
North Indian, Chinese 2357  
South Indian          1826  
Biryani               906  
Bakery, Desserts      906  
...  
North Eastern, Street Food 1  
Continental, North Indian, Chinese, Pizza, Thai 1  
Chinese, North Indian, Street Food, Rolls 1  
BBQ, Arabian, Chinese, Kebab, Beverages 1  
North Indian, Chinese, Arabian, Momos 1  
Name: count, Length: 2712, dtype: int64
```

cleaning

```
def cleaning_cuisine(val):  
    if (val=='North Indian, Chinese'):  
        return 'Multicuisine'  
    elif (val=='Bakery, Desserts'):  
        return 'Desserts'  
    elif (val=='Biryani'):  
        return 'Multicuisine'  
    elif ('Pizza' in str(val)):  
        return 'Italian'  
    elif (val=='North Indian'):  
        return 'North Indian'  
    elif (val=='South Indian'):  
        return 'South Indian'  
    else:  
        return 'Others'  
df['cuisines'].apply(cleaning_cuisine).value_counts()  
df['cuisines']=df['cuisines'].apply(cleaning_cuisine)
```

cek missing value

```
: print(df.isnull().sum())
```

```
name          0
online_order   0
book_table     0
rate           0
votes          0
rest_type      0
cuisines       0
costfor2       0
type           0
location       0
dtype: int64
```

cek data type

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 51382 entries, 0 to 51716
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   name             51382 non-null  object
1   online_order     51382 non-null  object
2   book_table       51382 non-null  object
3   rate             51382 non-null  float64
4   votes            51382 non-null  int64
5   rest_type        51382 non-null  object
6   cuisines         51382 non-null  object
7   costfor2         51382 non-null  float64
8   type             51382 non-null  object
9   location         51382 non-null  object
dtypes: float64(2), int64(1), object(7)
memory usage: 4.3+ MB
```

~ tidak ada missing value dan data type sudah benar, sehingga data sudah siap untuk digunakan.

statistika deskriptif

```
df.describe()
```

	rate	votes	costfor2
count	51382.000000	51382.000000	51382.000000
mean	3.701446	284.242303	555.433323
std	0.394906	804.886139	438.230133
min	1.800000	0.000000	40.000000
25%	3.500000	7.000000	300.000000
50%	3.700000	41.000000	400.000000
75%	3.900000	199.000000	700.000000
max	4.900000	16832.000000	6000.000000

statistika deskriptif untuk data kualitatif

```
df.describe(exclude='number')
```

	name	online_order	book_table	rest_type	cuisines	type	location
count	51382	51382	51382	51382	51382	51382	51382
unique	8758	2	2	8	6	7	30
top	Cafe Coffee Day	Yes	No	Quick Bites	Others	Delivery	BTM
freq	96	30244	44966	19096	40435	25740	3252

cleaned data

```
df
```

	name	online_order	book_table	rate	votes	rest_type	cuisines	costfor2	type	location
0	Jalsa	Yes	Yes	4.1	775	Casual Dining	Others	800.0	Buffet	Banashankari
1	Spice Elephant	Yes	No	4.1	787	Casual Dining	Others	800.0	Buffet	Banashankari
2	San Churro Cafe	Yes	No	3.8	918	Others	Others	800.0	Buffet	Banashankari
3	Addhuri Udipi Bhojana	No	No	3.7	88	Quick Bites	Others	300.0	Buffet	Banashankari
4	Grand Village	No	No	3.8	166	Casual Dining	Others	600.0	Buffet	Banashankari
...	...	...	...	...	...	...	...	...	...	...
51712	Best Brews - Four Points by Sheraton Bengaluru...	No	No	3.6	27	Others	Others	1500.0	Pubs and bars	Whitefield
51713	Vinod Bar And Restaurant	No	No	3.7	0	Others	Others	600.0	Pubs and bars	Whitefield
51714	Plunge - Sheraton Grand Bengaluru Whitefield H...	No	No	3.7	0	Others	Others	2000.0	Pubs and bars	Whitefield
51715	Chime - Sheraton Grand Bengaluru Whitefield Ho...	No	Yes	4.3	236	Others	Others	2500.0	Pubs and bars	Whitefield
51716	The Nest - The Den Bengaluru	No	No	3.4	13	Others	Others	1500.0	Pubs and bars	Whitefield

51382 rows × 10 columns

Research Questions

1. Does the demography of an area matters?
2. Does location of a particular type of restaurant also depends on the people living in that area?
3. Does the theme of the restaurant matters?
4. Is a food chain category restaurant likely to have more customers than its counter part?
5. Are any neighborhoods similar based on the type of food?
6. Is a particular neighborhood famous for its own kind of food?
7. If two neighborhoods are similar does that mean these are related or particular group of people live in the neighborhood or these are the places to it?
8. What kind of a food is more popular in a locality?
9. Do the entire locality loves vegetarian food. If yes then is that locality populated by a particular sect of people for eg. Jain, Marwaris, Gujaratis who are mostly vegetarian.

Does the demography of an area matters?

Methodology

Untuk menganalisis apakah demografi dari suatu area penting terhadap preferensi dan rating restoran, perlu dilakukan beberapa hal ini, termasuk: memastikan data pada kolom 'location', 'rate', dan 'votes' sudah bersih dari missing value; kemudian melakukan group data by location untuk menghitung rata-rata dari 'rate' dan 'votes'; menghitung korelasi antara rata-rata dari rate dan votes; terakhir, dapat dilakukan visualisasi.

Code Implementation

```
# Group by Location
location_analysis = df.groupby('location')[['rate', 'votes']].mean().reset_index()
location_analysis.sort_values('rate', ascending=False, inplace=True)

# Correlation Analysis
correlation = location_analysis['rate'].corr(location_analysis['votes'])
print(f'Correlation between average ratings and votes: {correlation:.2f}')

# Visualization
plt.figure(figsize=(12, 6))
sns.scatterplot(data=location_analysis, x='votes', y='rate')
plt.title('Average Ratings vs. Average Votes by Location')
plt.xlabel('Average Votes')
plt.ylabel('Average Rating')
plt.grid()
plt.show()
```

Analysis and Results

Correlation between average ratings and votes: 0.83



Diperoleh nilai korelasi sebesar 0.83 dengan scatter plot cenderung ke arah positif (naik ke kanan).

Interpretation and Conclusion

Grafik di atas menunjukkan bahwa ada hubungan positif (naik ke kanan) antara 'location', 'rate', dan 'votes'. Hal tersebut didukung dengan perolehan nilai korelasi sebesar 0,83, dengan arti bahwa variabel tersebut memiliki korelasi yang tinggi antara satu dengan yang lainnya. Disimpulkan bahwa faktor demografi memengaruhi preferensi restoran.

Does location of a particular type of restaurant also depends on the people living in that area?

Methodology

Untuk menganalisa apakah pemilihan lokasi tipe restoran tertentu bergantung kepada orang-orang yang tinggal di area tersebut, dapat dilakukan menggunakan cross-tabulation atau tabel kontingensi. Tabel tersebut akan menunjukkan distribusi dari 'rest_type' untuk sepanjang lokasi berbeda. Beberapa metode yang digunakan, termasuk: memastikan adanya column 'rest_type' dan 'location' dalam dataframe; melakukan data pre-processing; cross-tabulation dan visualisasi.

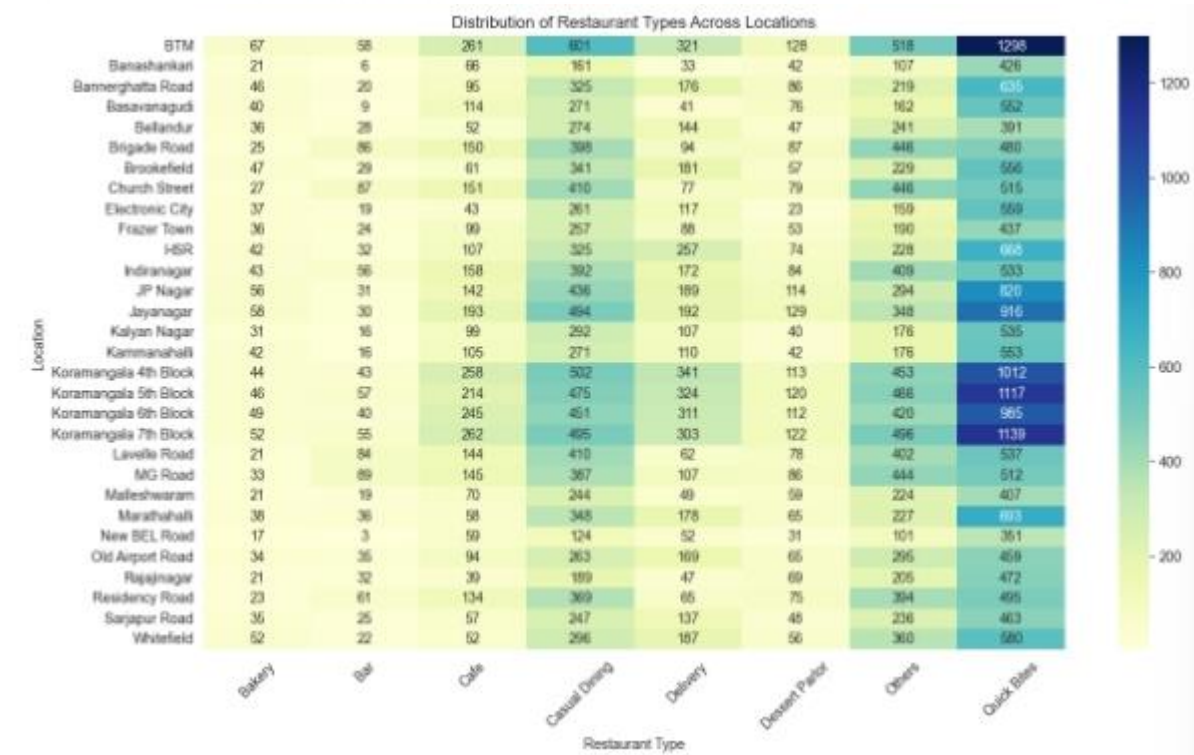
Code Implementation

```
# Create Cross-tabulation
location_rest_type_crosstab = pd.crosstab(df['location'], df['rest_type'])
print(location_rest_type_crosstab)

# Visualization
plt.figure(figsize=(14, 8))
sns.heatmap(location_rest_type_crosstab, annot=True, fmt='d', cmap='YlGnBu', cbar=True)
plt.title('Distribution of Restaurant Types Across Locations')
plt.xlabel('Restaurant Type')
plt.ylabel('Location')
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.show()
```

Analysis & Result

rest_type	Bakery	Bar	Cafe	Casual Dining	Delivery	\	rest_type	Dessert Parlor	Others	Quick Bites
location							location			
BTM	67	58	261	601	321		BTM	128	518	1298
Banashankari	21	6	66	161	33		Banashankari	42	107	426
Bannerghatta Road	46	20	95	325	176		Bannerghatta Road	86	219	635
Basavanagudi	40	9	114	271	41		Basavanagudi	76	162	552
Bellandur	36	28	52	274	144		Bellandur	47	241	391
Brigade Road	25	86	150	398	94		Brigade Road	87	446	480
Brookefield	47	29	61	341	181		Brookefield	57	229	556
Church Street	27	87	151	410	77		Church Street	79	446	515
Electronic City	37	19	43	261	117		Electronic City	23	159	559
Frazer Town	36	24	99	257	88		Frazer Town	53	190	437
HSR	42	32	107	325	257		HSR	74	228	668
Indiranagar	43	56	158	392	172		Indiranagar	84	409	533
JP Nagar	56	31	142	436	189		JP Nagar	114	294	820
Jayanagar	58	30	193	494	192		Jayanagar	129	348	916
Kalyan Nagar	31	16	99	292	107		Kalyan Nagar	40	176	535
Kammanahalli	42	16	105	271	110		Kammanahalli	42	176	553
Koramangala 4th Block	44	43	258	502	341		Koramangala 4th Block	113	453	1012
Koramangala 5th Block	46	57	214	475	324		Koramangala 5th Block	120	466	1117
Koramangala 6th Block	49	40	245	451	311		Koramangala 6th Block	112	420	985
Koramangala 7th Block	52	55	262	495	303		Koramangala 7th Block	122	496	1139
Lavelle Road	21	84	144	410	62		Lavelle Road	78	402	537
MG Road	33	89	145	387	107		MG Road	86	444	512
Malleshwaram	21	19	70	244	49		Malleshwaram	59	224	407
Marathahalli	38	36	58	348	178		Marathahalli	65	227	693
New BEL Road	17	3	59	124	52		New BEL Road	31	101	351
Old Airport Road	34	35	94	263	169		Old Airport Road	65	295	459
Rajajinagar	21	32	39	189	47		Rajajinagar	69	205	472
Residency Road	23	61	134	369	65		Residency Road	75	394	495
Sarjapur Road	35	25	57	247	137		Sarjapur Road	48	236	463
Whitefield	52	22	52	296	187		Whitefield	56	360	580



Tabel di atas merupakan tabulasi silang antara jenis restoran dan lokasi keberadaan, berisi jumlah restoran yang ada sesuai dengan lokasinya. Disajikan pula visualisasi heatmap agar pembaca dapat mengetahui informasi dengan lebih mudah.

Intpretation & Conclusion

Adanya visualisasi heatmap menambah keefektifan terjaringnya suatu informasi dibandingkan tabel biasa. Dapat dilihat dalam visualisasi tersebut, terdapat beberapa warna yang sudah dikategorikan. Semakin gelap warna, maka semakin tinggi pula jumlahnya. Pada jenis restoran 'bakery' pada setiap wilayah demografi tidak menunjukkan perbedaan yang terlihat jelas. Hal tersebut menunjukkan bahwa tipe restoran itu, tidak begitu bergantung pada demografi area, atau bisa tersebar di mana saja. Berbeda dengan tipe restoran 'casual dining' dan 'quick bites' yang memiliki penyebaran nilai cukup besar, dapat dengan mudah dilihat dari perbedaan warna yang sangat mencolok dan mudah dibedakan. Hal tersebut berarti untuk tipe restoran itu, bergantung pada jenis demografi lokasinya.

Does the theme of the restaurant matters?

Methodology

Untuk menganalisis apakah tema atau tipe dari restoran tersebut penting hubungannya dengan customer rating dan votes, maka dapat dilakukan perhitungan rata-rata rating dan votes untuk setiap tipe restaurant. Ada beberapa metode yang harus dilakukan, termasuk: memastikan adanya kolom 'rate' dan 'votes'; melakukan data pre-processing; melakukan perhitungan rata-rata dengan permulaan grouped by rest_type; dan visualisasi.

Code Implementation

```
# Calculate Average Ratings for Each Restaurant Type
average_ratings = df.groupby('rest_type')[['rate', 'votes']].mean().reset_index()

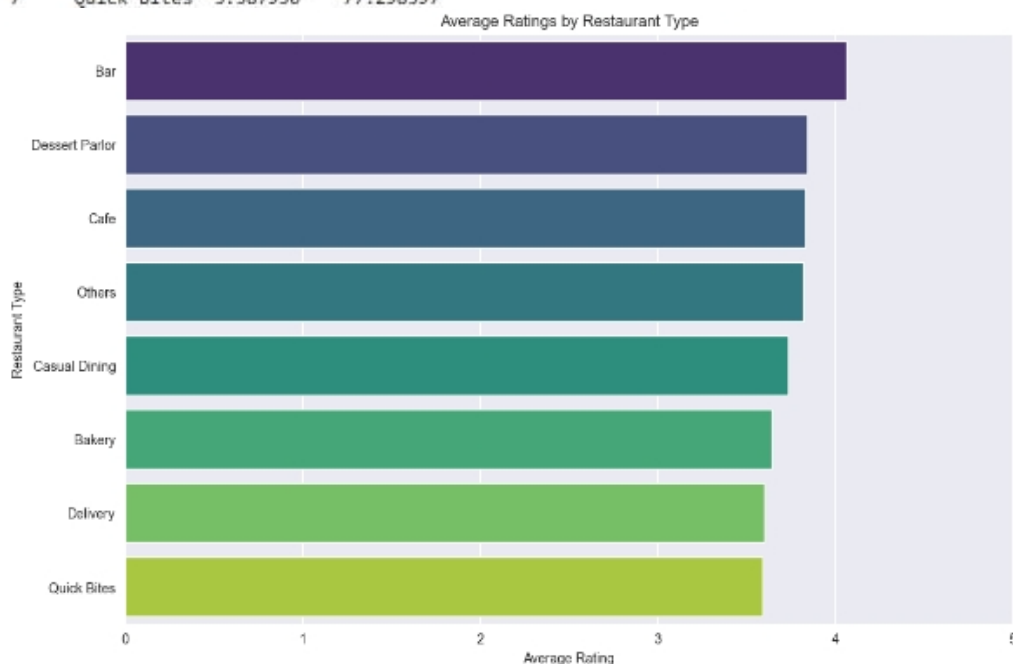
# Sort the DataFrame by average rating
average_ratings.sort_values('rate', ascending=False, inplace=True)

# Display the average ratings DataFrame
print(average_ratings)

# Step 4: Visualization
plt.figure(figsize=(12, 8))
sns.barplot(data=average_ratings, x='rate', y='rest_type', palette='viridis')
plt.title('Average Ratings by Restaurant Type')
plt.xlabel('Average Rating')
plt.ylabel('Restaurant Type')
plt.xlim(0, 5)
plt.show()
```

Analysis & Result

	rest_type	rate	votes
1	Bar	4.063676	1056.662892
5	Dessert Parlor	3.844248	142.735398
2	Cafe	3.834237	389.483499
6	Others	3.822390	579.071657
3	Casual Dining	3.737307	440.335144
0	Bakery	3.643158	31.214035
4	Delivery	3.600713	67.771540
7	Quick Bites	3.587950	77.298597



Tabel di atas merupakan tabel yang digunakan untuk menghitung rata-rata grouped by tipe restaurant. Ditampilkan sejumlah 8 baris jenis restaurant dengan masing-masing rerata rating dan voting yang kemudian divisualisasikan sebagai bar chart horizontal.

Interpretation & Conclusion

Grafik di atas menunjukkan distribusi nilai rata-rata antara 'rate' dan 'votes' untuk setiap kelompok tipe restaurant yang ada. Didapatkan informasi bahwa jenis restaurant berupa bar, memiliki jumlah peminat paling tinggi di antara lainnya. Hal tersebut dapat terjadi karena beberapa faktor, salah satunya tema restaurant yang dipilih. Jenis 'bar' mungkin saja dinilai sebagai tempat santai yang banyak diminati usia dewasa, sehingga mendorong customer untuk memberikan penilaian bagus.

Is a food chain category restaurant likely to have more customers than its counter part?

Methodology

Untuk menganalisa apakah suatu food chain restaurant mungkin punya lebih banyak pelanggan daripada restaurant independen lainnya, dapat membedakan dengan cara menandai atau mengkategorikan restaurant dengan nama yang sama. Kemudian dapat dibandingkan rata-rata keduanya. Ada beberapa cara dalam menganalisis, seperti: mengidentifikasi chain restaurant dengan menandai nama yang sama; grouped data by is_chain (True for chain restaurant, False for independen restaurant); melakukan perhitungan rata-rata rate dan vote; kemudian dapat dilakukan perbandingan dan visualisasi antar keduanya.

Code Implementation

```
# Create the 'is_chain' column
# A restaurant is considered a chain if its name appears more than once
df['is_chain'] = df['name'].duplicated(keep=False)

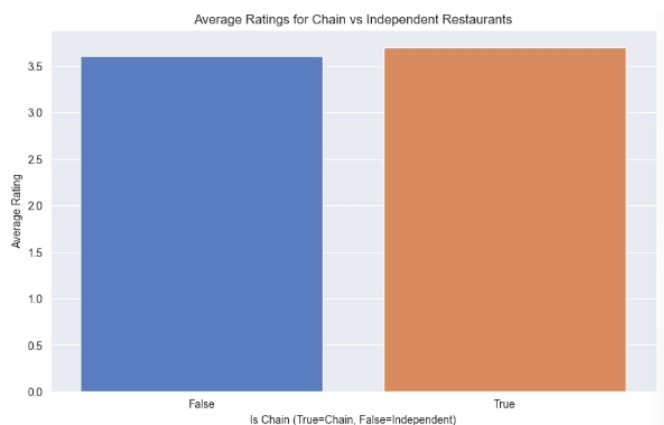
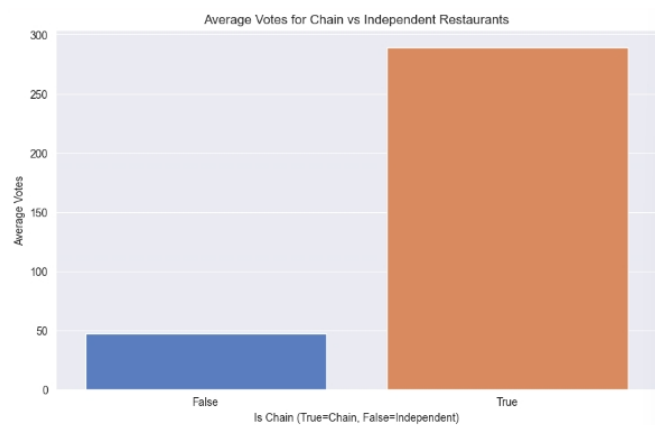
# Step 3: Group by 'is_chain' to calculate average ratings and votes
chain_analysis = df.groupby('is_chain')[['rate', 'votes']].mean().reset_index()
print(chain_analysis)

plt.figure(figsize=(10, 6))
sns.barplot(data=chain_analysis, x='is_chain', y='votes', palette='muted')
plt.title('Average Votes for Chain vs Independent Restaurants')
plt.xlabel('Is Chain (True=Chain, False=Independent)')
plt.ylabel('Average Votes')
plt.show()

plt.figure(figsize=(10, 6))
sns.barplot(data=chain_analysis, x='is_chain', y='rate', palette='muted')
plt.title('Average Ratings for Chain vs Independent Restaurants')
plt.xlabel('Is Chain (True=Chain, False=Independent)')
plt.ylabel('Average Rating')
plt.show()
```

Analysis & Result

	is_chain	rate	votes
0	False	3.605478	46.867224
1	True	3.703501	289.324361



Interpretation & Conclusion

Diberikan informasi detail mengenai rata-rata dari rate dan vote pada tabel grouped by is_chain. Dalam tabel tersebut diketahui bahwa untuk 'chain' restaurant (True) memiliki nilai rata-rata yang hampir mirip dengan rata-rata restaurant independen biasa. Berbeda dengan 'rate', untuk 'vote' diperoleh informasi bahwa customer lebih banyak memilih pada chain

restaurant dibanding lainnya. Hal tersebut divisualisasikan ke dalam dua bar chart vertikal di atas. Dengan membandingkan rata-rata dari 'votes', didapatkan kesimpulan bahwa food chain restaurant dapat memiliki lebih banyak pelanggan (higher votes) dibandingkan restoran lainnya.

Are any neighborhoods similar based on the type of food?

Methodology

Untuk menentukan apakah ada lingkungan atau lokasi serupa berdasarkan jenis makanan, dapat menggunakan pendekatan sebagai berikut: melakukan data preparation; melakukan teknik clustering, seperti K-means; menganalisis cluster setiap grup; identifikasi lingkungan yang sama; dan visualisasi.

Code Implementation

```
import pandas as pd
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Step 1: Explode the 'cuisines' column
df_exploded = df.explode('cuisines')

# Step 2: One-Hot Encode the Cuisines
mlb = MultiLabelBinarizer()
cuisines_encoded = mlb.fit_transform(df_exploded['cuisines'])
cuisines_df = pd.DataFrame(cuisines_encoded, columns=mlb.classes_)

# Step 3: Combine with original DataFrame
features = pd.concat([df_exploded[['location']], cuisines_df], axis=1)

# Step 4: Group by location and sum the one-hot encoded cuisines
features_grouped = features.groupby('location').sum()

# Step 5: Standardize the features
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features_grouped)

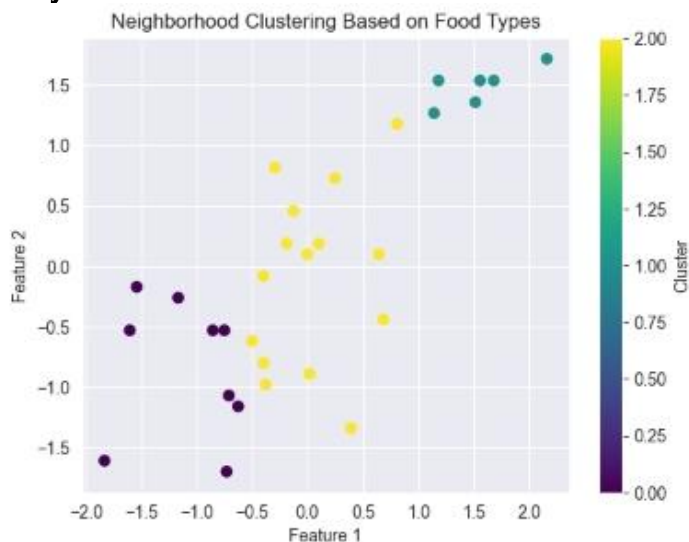
# Step 6: Apply K-Means Clustering
kmeans = KMeans(n_clusters=3) # Adjust the number of clusters
features_grouped['cluster'] = kmeans.fit_predict(features_scaled)

# Step 7: Analyze Clusters
cluster_analysis = features_grouped.groupby('cluster').agg(lambda x: ', '.join(x.index[x > 0])).reset_index()

# Step 8: Visualization (Optional)
plt.scatter(features_scaled[:, 0], features_scaled[:, 1], c=features_grouped['cluster'], cmap='viridis')
plt.title('Neighborhood Clustering Based on Food Types')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.colorbar(label='Cluster')
plt.show()

# Print cluster analysis
print(cluster_analysis)
```

Analysis & Result



```

cluster
0      0 Banashankari, Electronic City, Frazer Town, Ka...
1      1 BTM, Jayanagar, Koramangala 4th Block, Koraman...
2      2 Bannerghatta Road, Basavanagudi, Bellandur, Br...

0 Banashankari, Electronic City, Frazer Town, Ka...
1 BTM, Jayanagar, Koramangala 4th Block, Koraman...
2 Bannerghatta Road, Basavanagudi, Bellandur, Br...

```

Analisis cluster akan menunjukkan lingkungan yang grouped by cluster dan masakan. Informasi tersebut nantinya dapat digunakan untuk melihat kemiripan lingkungan yang memiliki kesamaan preferensi cuisine.

Interpretation & Conclusion

Grafik tersebut menunjukkan analisis clustering berdasarkan jenis makanan di beberapa lingkungan. Terdapat tiga kelompok (cluster) yang diwakili oleh warna ungu, kuning, dan hijau. Setiap kelompok menggambarkan lingkungan yang memiliki preferensi makanan serupa, di mana lingkungan dalam cluster yang sama menunjukkan kesamaan dalam jenis makanan yang dominan. Semakin dekat jarak titik, maka mengindikasikan lingkungan di dalam cluster memiliki kemiripan tipe makanan.

Is a particular neighborhood famous for its own kind of food?

Methodology

Untuk menganalisa apakah suatu lingkungan tertentu memiliki makanan terkenal mereka sendiri, dapat dilakukan pendekatan sebagai berikut: data preparation, menentukan cuisine paling populer, identifikasi lingkungan yang memiliki jumlah cuisine tertentu paling banyak; dan visualisasi.

Code Implementation

```

# Explode the 'cuisines' column
df_exploded = df.explode('cuisines')

# Count occurrences of each cuisine by neighborhood
cuisine_counts = df_exploded.groupby(['location', 'cuisines']).size().reset_index(name='count')

# Find the most popular cuisine in each neighborhood
popular_cuisines = cuisine_counts.loc[cuisine_counts.groupby('location')['count'].idxmax()]

# Create a DataFrame of maximum counts for each location
max_counts = cuisine_counts.groupby('location')['count'].max().reset_index()
max_counts.columns = ['location', 'max_count'] # Rename for clarity

# Merge to get max_count with popular_cuisines
famous_cuisines = popular_cuisines.merge(max_counts, on='location')

# Define a threshold for "famous" cuisine (it appears at least twice as much as the second most popular)
threshold = 2 # Adjust this threshold based on your preference

# Find neighborhoods where the most popular cuisine meets the threshold
famous_cuisines['is_famous'] = famous_cuisines['count'] > (famous_cuisines['max_count'] / threshold)

# Filter for neighborhoods with a famous cuisine
famous_neighborhoods = famous_cuisines[famous_cuisines['is_famous']]
print(famous_neighborhoods[['location', 'cuisines', 'count']])

```

Analysis & Result

	location	cuisines	count
0	BTM	Others	2608
1	Banashankari	Others	622
2	Bannerghatta Road	Others	1270
3	Basavanagudi	Others	946
4	Bellandur	Others	888
5	Brigade Road	Others	1444
6	Brookefield	Others	1120
7	Church Street	Others	1463
8	Electronic City	Others	941
9	Frazer Town	Others	938
10	HSR	Others	1345
11	Indiranagar	Others	1458
12	JP Nagar	Others	1640
13	Jayanagar	Others	1870
14	Kalyan Nagar	Others	1057
15	Kammanahalli	Others	1065
16	Koramangala 4th Block	Others	2198
17	Koramangala 5th Block	Others	2230
18	Koramangala 6th Block	Others	2073
19	Koramangala 7th Block	Others	2302
20	Lavelle Road	Others	1395
21	MG Road	Others	1471
22	Malleswaram	Others	868
23	Marathahalli	Others	1222
24	New BEL Road	Others	570
25	Old Airport Road	Others	1146
26	Rajajinagar	Others	846
27	Residency Road	Others	1308
28	Sarjapur Road	Others	917
29	Whitefield	Others	1214

Gambar kedua menampilkan hasil analisis, mencantumkan lingkungan dan masakan mereka yang paling populer bersama dengan jumlahnya.

Interpretation & Conclusion

Sebagian besar lingkungan seperti BTM, Banashankari, Bannerghatta Road, dan lainnya memiliki kategori "Others" sebagai masakan yang paling umum, yang menunjukkan bahwa tidak ada jenis masakan tertentu yang secara signifikan lebih populer di lingkungan-lingkungan ini berdasarkan dataset ini. Analisis ini menyimpulkan bahwa tidak ada masakan spesifik yang menonjol sebagai terkenal di sebagian besar lingkungan akibat dominasi kategori "Others".

If two neighborhoods are similar does that mean these are related or particular group of people live in the neighborhood or these are the places to it?

Methodology

Data diolah dengan menggunakan teknik *pivot table* untuk membentuk sebuah matriks yang merepresentasikan rata-rata rating setiap tipe restoran di berbagai lokasi atau lingkungan (neighborhood). Data yang dipivot menghasilkan sebuah tabel di mana lokasi ditetapkan sebagai indeks, jenis restoran sebagai kolom, dan nilai rata-rata rating sebagai isian. Nilai yang hilang diisi dengan angka 0 untuk menjaga konsistensi data.

Code Implementation

```
# nomor 7
neighborhood_similarity = df.pivot_table(index='location', columns='rest_type', values='rate', aggfunc='mean').fillna(0)
neighborhood_similarity
```

Analysis & Result

	rest_type	Bakery	Bar	Beverage Shop	Cafe	Casual Dining	Delivery	Dessert Parlor	Food Court	Others	Quick Bites
location											
BTM	3.677612	4.064762	3.615278	3.868582	3.755408	3.575078	3.921875	3.531915	3.879545	3.601387	
Banashankari	3.714286	3.400000	3.836364	3.748485	3.796646	3.660606	3.697619	0.000000	3.695238	3.634507	
Bannerghatta Road	3.654348	4.031818	3.993939	3.672632	3.589846	3.617045	3.677907	3.487234	3.648905	3.543465	
Basavanagudi	3.695000	3.555000	3.833333	3.857018	3.759410	3.658537	3.831579	3.683333	3.724194	3.632790	
Bellandur	3.600000	4.170270	3.623529	3.751923	3.581387	3.643056	3.831915	3.451351	3.847191	3.524552	
Brigade Road	3.552000	3.971875	3.661905	3.831333	3.803266	3.626596	3.893103	3.600000	3.978437	3.588125	
Brookfield	3.619149	3.709091	3.600000	3.598361	3.595894	3.572376	3.717544	3.485714	3.738012	3.553777	
Church Street	3.551852	3.985271	3.652941	3.853642	3.799512	3.609091	3.868354	3.609091	3.992553	3.589126	
Electronic City	3.500000	3.599091	3.708333	3.483721	3.510945	3.585470	3.543478	3.630000	3.746429	3.552594	
Frazer Town	3.747222	3.982353	3.611765	3.834343	3.801556	3.579545	3.790566	3.680000	3.890196	3.567277	
HSR	3.695238	4.000000	3.686364	3.827103	3.640908	3.612451	3.900000	3.425000	3.842105	3.587575	
Indiranagar	3.616279	3.855682	3.685185	3.767722	3.806378	3.591279	3.869048	3.600000	3.949853	3.572608	
JP Nagar	3.707143	4.021053	3.647727	3.739437	3.681422	3.602116	3.752632	3.541304	3.709645	3.575610	
Jayanagar	3.752759	3.873333	3.642105	3.803109	3.763158	3.580729	3.817829	3.535714	3.770940	3.625109	
Kalyan Nagar	3.525806	3.744828	3.639130	3.786869	3.711986	3.567290	3.820000	3.514286	3.798496	3.591776	
Kammanahalli	3.650000	3.687097	3.650000	3.773333	3.704428	3.555455	3.826190	3.477778	3.790625	3.582821	
Koramangala 4th Block	3.659091	4.025301	3.640299	3.955426	3.803187	3.598534	3.957522	3.680000	3.929762	3.628261	
Koramangala 5th Block	3.684783	4.001667	3.654688	3.974766	3.824000	3.610185	3.950000	3.550000	3.934347	3.621486	
Koramangala 6th Block	3.669388	4.020253	3.637097	3.970612	3.823503	3.609968	3.957143	3.655556	3.933871	3.621726	
Koramangala 7th Block	3.655769	4.039496	3.642424	3.956107	3.811919	3.615182	3.963934	3.515789	3.922767	3.616681	
Laville Road	3.538095	3.934711	3.620000	3.838889	3.775610	3.609677	3.891026	3.646154	3.977108	3.581192	
MG Road	3.600000	4.014961	3.590000	3.860690	3.831008	3.592523	3.890698	3.575000	3.993850	3.569141	
Malleswaram	3.752381	3.845161	3.520000	3.821429	3.790984	3.659184	3.786441	3.487879	3.785799	3.625307	
Marathahalli	3.547368	3.765957	3.640909	3.581034	3.582184	3.533708	3.700000	3.471875	3.791358	3.537807	
New BEL Road	3.576471	3.757143	3.700000	3.730508	3.621774	3.653846	3.812903	0.000000	3.875000	3.570370	
Old Airport Road	3.544118	3.829825	3.726316	3.706383	3.838783	3.592899	3.876823	3.525000	3.924390	3.559913	
Rajajinagar	3.742857	3.785366	3.530000	3.751282	3.778307	3.678723	3.685507	3.461111	3.712667	3.597034	
Residency Road	3.513043	4.011828	3.609091	3.816418	3.787805	3.629231	3.894667	3.642857	3.976074	3.578182	
Sarjapur Road	3.657143	4.100000	3.631579	3.782456	3.553846	3.621898	3.806250	3.292000	3.820765	3.510799	
Whitefield	3.671154	3.896585	3.654545	3.711538	3.706419	3.607487	3.710714	3.510606	3.824111	3.531552	

Setelah data dipivot, kita dapat membandingkan kesamaan antar lingkungan. Dari hasil analisis ini, dapat mengidentifikasi pola. Misalnya, lingkungan yang memiliki nilai kesamaan tinggi mungkin menunjukkan bahwa mereka menawarkan pengalaman bersantap yang serupa. Ini bisa berarti bahwa masyarakat di kedua lingkungan tersebut memiliki preferensi yang hampir sama dalam hal makanan dan minuman, atau jenis restoran yang berkembang di sana mirip.

Interpretation & Conclusion

Lingkungan yang menunjukkan kemiripan dalam jenis restoran dan ratingnya bisa dianggap memiliki keterkaitan dalam hal gaya hidup atau preferensi masyarakat terhadap restoran. Ini dapat berarti bahwa di lingkungan-lingkungan tersebut, orang-orang memiliki kecenderungan yang sama dalam memilih tempat makan, baik dari segi jenis makanan maupun pengalaman bersantap.

What kind of a food is more popular in a locality?

Methodology

Untuk mengetahui jenis makanan apa yang lebih populer di suatu lokasi, data diolah dengan cara mengelompokkan hidangan yang disukai (*dish_liked*) berdasarkan lokasi. Data kemudian dihitung untuk mengetahui seberapa sering hidangan tersebut disukai di setiap lokasi. Hasilnya diurutkan untuk mendapatkan hidangan paling populer di tiap lokasi.

Code Implementation

```
liked_dishes_counts = df.groupby(['location', 'dish_liked']).size().reset_index(name='count')

top_n = 3
top_liked_dishes = liked_dishes_counts.sort_values(['location', 'count'], ascending=[True, False])
top_liked_dishes = top_liked_dishes.groupby('location').head(top_n)

# Output the results
print(top_liked_dishes)
```

Analysis & Result

	location	dish_liked	count
10	BTM	Biryani	24
125	BTM	Masala Dosa	21
157	BTM	Paneer Tikki, Mutton Raan, Mango Margarita, Cr...	17
302	Banashankari	Pasta, Gelato, Garlic Bread, Mojito, Nachos, P...	13
254	Banashankari	Churros, Cannelloni, Minestrone Soup, Hot Choc...	10
...	...	...	...
5536	Wilson Garden	Beer	5
5542	Yelahanka	Appam, Fish Curry, Fish Fry, Pothichoru	2
5544	Yeshwantpur	Badam Milk, Chaat, Sandwich, Noodles, Ghee Dos...	2
5545	Yeshwantpur	Bamboo Chicken Biryani	2
5546	Yeshwantpur	Beef Biryani, Paratha, Fish Curry, Chicken Bir...	2

[247 rows x 3 columns]

Dari hasil pemrosesan ini, dapat dilihat ada beberapa hidangan tertentu mendominasi di lokasi tertentu. Sebagai contoh:

- Di BTM, hidangan seperti *Biryani* dan *Masala Dosa* menempati posisi teratas.
- Di Banashankari, makanan seperti *Pasta* dan *Gelato* lebih sering disukai.

Jenis makanan yang disukai ini menunjukkan preferensi unik di tiap lokasi yang mungkin disebabkan oleh demografi penduduk atau ketersediaan restoran dengan spesialisasi tertentu.

Interpretation & Conclusion

Hasil di atas menunjukkan bahwa setiap lokasi memiliki preferensi makanan yang berbeda-beda. Misalnya, di daerah BTM, hidangan klasik India seperti *Biryani* dan *Masala Dosa* sangat populer, menunjukkan bahwa kawasan ini mungkin dihuni oleh orang-orang yang lebih menyukai hidangan tradisional India. Sebaliknya, di Banashankari, preferensi makanan lebih beragam dengan kehadiran makanan internasional seperti *Pasta* dan *Gelato*, yang mungkin menandakan adanya populasi yang lebih kosmopolitan atau kehadiran lebih banyak restoran internasional di sana.

Do the entire locality loves vegetarian food. If yes then is that locality populated by a particular sect of people for eg. Jain, Marwaris, Gujaratis who are mostly vegetarian.

Methodology

Untuk mengetahui apakah seluruh lokasi lebih menyukai makanan vegetarian, perlu dilakukan analisis data terkait dengan makanan yang disukai (*dish_liked*) di setiap lokasi. Langkah awal adalah mengidentifikasi makanan vegetarian yang disukai dan menghitung frekuensi makanan vegetarian dibandingkan dengan non-vegetarian di setiap lokasi.

~ belum diketahui detail informasi lebih lanjut mengenai makanan vegetarian, sehingga belum bisa dilanjutkan.